



GitMatch : Developer-Issue Matching Platform

Mr. Yuvraj J. Jadhav

Assistant Professor, Computer Science & Engineering, Dnyanshree Institute of Engineering & Technology, Satara.

Mr. Kaustubh Pawar, Mr. Purvesh Shinde, Mr. Amey Gawade, Mr. Ganesh Najan

Student, Department of Computer Science & Engineering, Dnyanshree Institute of Engineering & Technology, Satara.

Abstract— One of the main challenges for software developers finding the right open-source projects for them is a difficult one that will cause lowered productivity, unused skill sets etc. To solve this we created GitMatch AI a web application that intelligently matches software developers with GitHub issues and projects, the model matches contribution history, preferred programming languages, skills and even behavior without needing to perform skill assessments, integrated with Google Cloud Vertex AI it is now easy to match developers with the appropriate project. The framework consists of React.js, Node.js, Express.js and MongoDB enabling efficient performance. GitMatch AI helps with open-source contribution quality, making open-source collaboration an easier and more enjoyable task for developers while also allowing for a more useful and educational experience.

Keywords:- AI-Powered Recommendation System, Developer Profiling, GitHub Issue Recommendation, Open-Source Contribution, Software Collaboration Platform

I. INTRODUCTION

The modern world of software development is incredibly fast-paced, and with this, collaborative, open-source development is an integral part of learning and creativity. However, finding a GitHub issue or project that interests you and suits your skills, can be frustrating for developers, causing them to lose time, not be productive, and their skills be of waste. GitMatch AI's AI based system, finds opportunities for developers on open source projects to match skills, interests, using the GitHub API to find opportunities. Through the Google Cloud Vertex AI, the system analyses skills, history of development and coding styles, then determines the best projects/issues that fit you. There is a built-in messaging feature to allow developers to contact, and work with each other, all within one place. .

1.1 Developer Profiling System :

One of the critical modules is the Developer Profiling System in GitMatch AI. The module's primary responsibility is to learn and understand about a developer's technical background and ability to contribute. The system extracts information including repositories and commit history, programming languages, contributor frequency, topics the developer's project are involved with, as well as patterns in

coding behaviors through the GitHub API. Once this data is extracted, it is analyzed in order to produce a well-formed developer profile which captures the individual's technical skill sets and preferences. While current platforms allow users to list out the skills they are capable of using for contribution, GitMatch AI assesses skills through an analysis of contributions to GitHub in real-time and is therefore more objective and accurate.

The profiling system ultimately serves the function of mitigating mismatch between developers and open-source projects by pointing out which technologies a contributor has more experience and activity on. This helps in finding the right issues to be recommended and avoids wasted time from both the developer and the maintainer of an open-source project. Furthermore, this module is constantly updating a contributor's profile with current data, thus it is always up-to-date and relevant. Thus, the Developer Profiling System forms the core of GitMatch AI by making a more objective analysis of developers possible

1.2 AI-Based Recommendation Engine :

The Recommendation Engine is the central module in GitMatch AI that is responsible for making relevant recommendations between developers and issues on GitHub. The engine utilizes Google Cloud Vertex AI to examine and match developer profiles with open-source opportunities available. The recommendations are determined by various aspects, including programming languages used, topic tags of the repositories, coding styles followed in code repositories, contribution history of developers, difficulty of issues to be addressed and developer preferences. Such analysis provides developers with relevant projects they could work on as well as helps them choose projects according to their skills and interest in learning. The Recommendation Engine minimizes the manual efforts required to search relevant issues and project repositories on GitHub and enhances the possibility of successfully contributing to an issue. It also helps maintainers in the community to attract skilled developers for their project. It also improves recommendations based on user responses, issues that they chose to work on and previous work they had committed.

1.3 Decoupled Application Framework and Collaborative Interface:

The profiling and recommendation modules are supported by a strong, decoupled MERN (MongoDB, Express.js, React.js and Node.js) technology stack that the GitMatch AI is based upon. The front-end of the platform is developed using React.js, and provides developers with a light-weight and interactive UI where they can easily view their suggested task list, skill insights and repositories. The tasks are handled by Node.js and Express on the back end, which is used for secure authentication of the GitHub API, routes communication, and to communicate with the intelligence behind Vertex AI. MongoDB provides a flexible NoSQL data storage, and it is best suited for saving more unstructured and semi-structured data like dynamic developer profile histories, and the historical task list, among others. The modularity of this framework not only connects contributors and project maintainers, but it also provides a unified working environment that speeds up developers' learning process, and the development of open-source projects.

II. LITERATURE SURVEY

Early approaches to solving the developer-issue mismatch focused on repository-level recommendations. Surbhi Sharma and Anuj Mahajan (2017) reviewed various suggestive approaches for creating a recommender system for GitHub. Their work analyzed the efficacy of collaborative filtering, content-based, hybrid, and knowledge-based approaches aimed at suggesting relevant repositories to GitHub users. While effective at the repository level, these models often lack the granularity required to assign specific *issues* based on a developer's dynamic technical capabilities

Understanding the health and lifecycle of open-source projects is crucial for building effective recommendation algorithms. Abin Joy, Senthilkumar Thangavelu, and Amalendu Jyotishi (2018) conducted an empirical analysis evaluating the performance and health of GitHub OSS projects. By analyzing contributor metrics, growth patterns, and engagement levels, their research provides a foundational data-driven perspective on project dynamics. This emphasizes the need for recommendation systems to factor in not just developer skills, but the scale and activity level of the target repository [1].

Shifting the focus from suggesting repositories to suggesting *contributors*, Xunhui Zhang, Tao Wang, Gang Yin, Cheng Yang, and Huaimin Wang (2017) introduced the "ConRec" approach. This model utilizes a combination of developer historical activity, collaborative filtering, and text matching to recommend suitable contributors for open-source projects. This represented a significant step forward by utilizing historical behavioral data rather than relying solely on manually declared skills [2].

More recent literature has increasingly adopted advanced machine learning to personalize task assignments and overcome the "cold-start" problem for newcomers. For instance, Xiao et al. (2023) developed *PFIRec* (Personalized First Issue Recommender), which utilizes LambdaMART algorithms to match newcomers to "Good First Issues" based on sentiment, expertise preference, and previous OSS experience. Furthermore, modern systematic reviews on Crowdsourced Software Engineering (CSE) highlight that newer systems must

aggressively leverage Natural Language Processing (NLP) to parse issue descriptions and match them contextually with developer profiles, rather than relying on keyword similarity alone [3].

While traditional collaborative filtering and metadata analysis have laid the groundwork (Sharma & Mahajan, 2017; Zhang et al., 2017), current systems still struggle with the inflexibility of static queries and high mismatch rates. Many approaches require manual skill input or rely on simplistic keyword matching. GitMatch AI addresses the specific gaps identified in this literature. By dynamically extracting contributor data through the GitHub API and processing contextual embeddings via Google Cloud Vertex AI, GitMatch AI bypasses the limitations of manual profiling [4].

Further advancing the use of AI in repository management, recent research has heavily explored Natural Language Processing (NLP) for automated bug triaging and issue assignment. Studies leveraging deep learning algorithms have demonstrated high accuracy in extracting semantic meaning from unstructured, user-generated issue descriptions. By mapping the textual requirements of an issue directly to the historical text footprints of developers' past commits, these models prove that semantic similarity outperforms traditional keyword tracking.[5]

Comparison on Existing Technology

Table 1. Comparison Table

Parameters (↓)	Github Recommen dation Systems	Traditional Issue Assignment Systems	GitMatch Ai
Designing approach	Collaborati ve filtering approach	Manual assignment strategy	Ai-based similarity matching approach
Classificati on	Keyword- based recommen dation	Rule-based assignment	NLP and Ai-based recommen dation
Chance of mismatch	High chance of mismatch	Moderate chance	Less chance
Type of system	Reactive system	Semi- automated system	Intelligent proactive system
Real-Time Recommen dation	Limited support	Not available	Real-time issued

III. PROPOSED APPROACH FRAMEWORK AND DESIGN

The architectural design of the proposed intelligent developer-issue matchmaking system (GitMatch AI) is conceptually presented in Fig. 3 below. The proposed system consists of four sequential operational phases: authentication & request validation, developer data ingesting & real-time processing, recommendation engine execution, similarity based ranking & delivery of relevant tasks. The platform uses technical features to produce individualized open-source task recommendations.

3.1 Developer UI Dashboard (Client Interface):

The client interface is used for the direct consumption of the system by the end-users. Built using React.js, it's a thin and interactive dashboard where developers log in to secure GitHub OAuth authentication sessions and can effortlessly access their respective task lists. Developers' interaction can be logged and presented to an optimized dashboard with aggregated data, language interests, automated skills insight without a strenuous manual profile verification process.

3.2 Decoupled Web and Storage Server:

The execution layer: which is the backbone of the application, like the brain. It hosts the platform logic and manages the data storage. This server is built on top of Node.js and Express, and handles the securely transmitted session token, routes the APIs, and handles the requests made to the machine learning layers.

3.3 GitMatch (Server):

Automated endpoints and semantics are incorporated into the core middleware layer to resolve the inflexibility of static, keyword-driven queries. To begin with, automatically populated endpoints are ingested from a contributor's public profile through the GitHub REST API and keyed according to commit activity trends and repository organization. These ingested targets are then standardized and processed sequentially through the Vertex AI intelligent analysis of Google Cloud, gaining context. Then, complex NLP models are leveraged to generate contextual feature embeddings from current GitHub issue descriptions, as well as a vector of a contributor profile, in the matching engine.

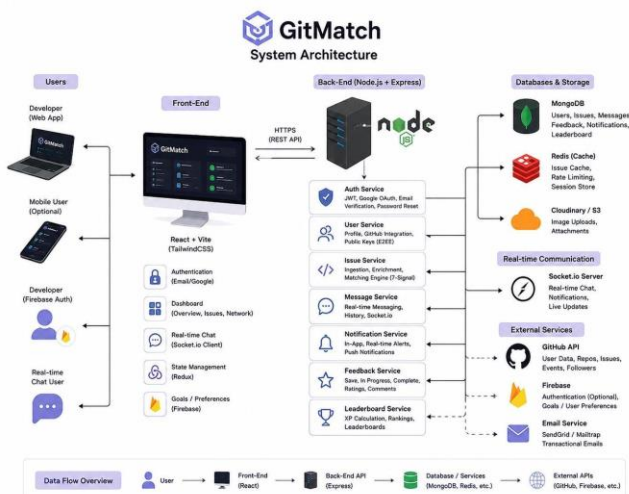


Fig.1 System Architecture

IV. ALGORITHM :

4.1 ISSUE MATCHING AND RANKING

ALGORITHM :

Input:

U – User Profile Vector
E – Set of Enriched Issues
F – Feedback Map

Output:

R – Ranked Recommended Issues

Initialize: $R \leftarrow \{\}$, Scores $\leftarrow \{\}$, Penalties $\leftarrow \{\}$

Repeat

For each issue $e \in E$ do

if $F[e.id]$ contains “hide” then Continue

skillMatch $\leftarrow$ computeSkillMatch(U.skills, e.requiredSkills)

difficultyFit $\leftarrow$ computeDifficultyFit(U.meta.seniorityTier, e.difficultyScore)

issueTypeFit $\leftarrow$ computeIssueTypeFit(U.meta.preferredIssueTypes, e.issueType)

repoScaleFit $\leftarrow$ computeRepoScaleFit(U.meta.preferredRepoScale, e.repo.scale)

overall $\leftarrow$ weighted sum of all scores

matchedSkills $\leftarrow$ intersect(U.skills, e.requiredSkills)

avgConfidence $\leftarrow$ average confidence of matchedSkills

overall $\leftarrow$ overall $\times$ avgConfidence

penalties $\leftarrow$ computePenalties(e, $F[e.id]$)

overall $\leftarrow$ clamp(overall + penalties.total, 0, 1)

if overall ≥ 0.05 then

$R \leftarrow R + \{issue:e, matchScore:overall\}$

end if

End For

Sort R by:

1. Highest Match Score
2. Fewer Assignees
3. Recently Created Issues

Return R

4.2 MATHEMATICAL MODEL

System = {S, T, E, X, Y, Fmain, DD, NDD, SWr, HWr, AImodel, DB, API} S = Initial State

S = {Developer Profile, GitHub Repository Data, User Preferences, Database} E = End State

E = {Ranked GitHub Issue Recommendations}

X = Inputs

X = {I1, I2, I3}

I1 = GitHub User Data

I2 = Repository and Issue Data

I3 = Developer Skill Information

Y = Outputs

Y = {O1, O2}

O1 = Personalized Issue Recommendations

O2 = Skill Matching and Ranking Results

T = Number of Steps Performed

T = 6

DD = Deterministic Data

DD = {Repositories, Issues, Skills, Commit History}

NDD = Non Deterministic Data

NDD = {User Preferences, Repository Activity, Contribution Patterns}

SWr = Software Requirements

HWr = Hardware Requirements

AImodel = Artificial Intelligence Recommendation Engine

DB = MongoDB Database

API = GitHub REST API and Vertex AI API

Fmain = Main Function of the system is to analyze developer skills, GitHub contribution history, repository activities, and issue descriptions to intelligently recommend the most suitable open-source GitHub issues using AI-based similarity matching and ranking techniques.

A. Hardware and software specifications:

- Hardware Specifications
System (Min) – Intel i5
Processor
RAM – 8 GB
- Software Specifications
OS – Windows 10 or
Higher

Technology – MERN Stack,

Vertex AI

Frontend – React.js

Backend – Node.js, Express.js

Database – MongoDB

Tools – VS Code, GitHub

API, Firebase

V. RESULTS AND DISCUSSION

GitMatchAI is an intelligent and efficient system that finds suitable open-source GitHub issues for developers through AI-based skills analysis and similarity ranking methods. It enhances contribution efficiency, diminishes manual issue searching, and strengthens co-operation between contributor and project maintainer.

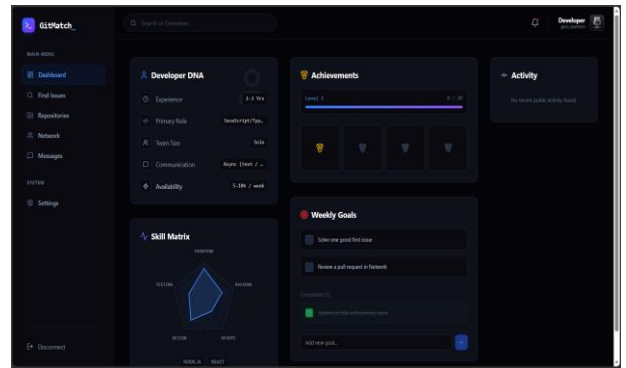


Fig.2 GitMatch -Dashboard

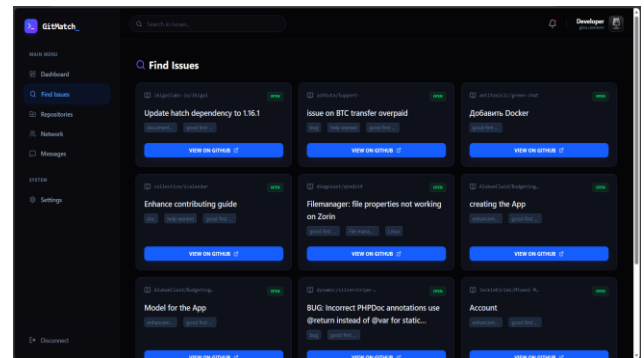


Fig.3 GitMatch Issues

VI. CONCLUSION

GitMatch is a smart AI-based solution that enables efficient matching of developers with suitable GitHub issues considering skill set, history of coding and their preferences. The tool helps boost the open-source community by saving human effort and effort of issue discovery while elevating the quality of contributions.

VII. REFERENCE

- [1] Y. Koren, R. Bell, and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," *Computer*, vol. 42, no. 8, pp. 30–37, IEEE, 2009.
- [2] D. M. German, "The GNOME Project: A Case Study of Open Source, Global Software Development," *Software Process: Improvement and Practice*, vol. 8, no. 4, pp. 201–215, 2003.
- [3] I. Steinmacher, M. A. Gerosa, and D. Redmiles, "Attracting, Onboarding, and Retaining Newcomer Developers in Open Source Software Projects," presented in the *Workshop on Global Software Development in a CSCW Perspective*, 2014.
- [4] F. Canfora, M. D. Penta, and L. Cerulo, "Achievements and Challenges in Software Reverse Engineering," *Communications of the ACM*, vol. 54, no. 4, pp. 142–151, 2011.
- [5] C. Bird, P. Rigby, E. Barr, D. Hamilton, D. German, and P. Devanbu, "The Promises and Perils of Mining Git," presented in the *6th IEEE International Working Conference on Mining Software Repositories*, 2009.

- [6] T. Fritz and G. Murphy, "Using Information Fragments to Answer the Questions Developers Ask," presented in *ICSE 2010 Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering*, 2010.
- [7] A. Begel and T. Zimmermann, "Analyze This! 145 Questions for Data Scientists in Software Engineering," presented in the *36th International Conference on Software Engineering*, 2014.
- [8] I. Steinmacher, T. Conte, M. Gerosa, and D. Redmiles, "Social Barriers Faced by Newcomers Placing Their First Contribution in Open Source Software Projects," presented in the *18th ACM Conference on Computer Supported Cooperative Work & Social Computing*, 2015.
- [9] S. Baltes and P. Diehl, "Towards a Theory of Software Development Expertise," presented in the *2018 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, 2018.
- [10] T. Mens and A. Serebrenik, "The Evolution of Social Ecosystems in Open Source Software Communities," presented in the *International Conference on Software Maintenance*, 2011.
- [11] A. Bacchelli and C. Bird, "Expectations, Outcomes, and Challenges of Modern Code Review," presented in the *35th International Conference on Software Engineering*, 2013.